(CSBB-311)

PROJECT REPORT

SMART CROWD DENSITY MONITOR

(A Machine Learning Based Density Monitoring System)

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE & ENGINEERING

Submitted By

Mayank Raj (241210076)

Md Yahiya Ariz (241210077)

Sahil Dindwal (241210102)

Yash Chawliya (241210135)

Submitted To

Dr. Gautam Kumar (Project Coordinator)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

NATIONAL INSTITUTE OF TECHNOLOGY DELHI

NOVEMBER 2026

CERTIFICATE

This is to certify that the project report entitled “Smart Crowd Density Monitoring for Public Safety Using Deep Learning (CSRNet)” submitted by Mayank Raj, Md Yahiya Ariz, Sahil Dindwal and Yash Chawliya to the Department of Computer Science and Engineering, National Institute of Technology Delhi, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of project work carried out by the students under my supervision and guidance.

To the best of my knowledge, the work presented in this report is an academic project record. Any external datasets, research papers, software libraries and other resources used have been acknowledged in the report.

Project Guide	Head of Department
Signature: _____	Signature: _____
Date: _____	Date: _____

DECLARATION

We hereby declare that the project work entitled "Smart Crowd Density Monitoring for Public Safety Using Deep Learning (CSRNet)" is an academic project carried out by us under the guidance of Dr. Gautam Kumar in the Department of Computer Science and Engineering, National Institute of Technology Delhi. The report describes the implementation and experiments recorded in the submitted project notebook. To the best of our knowledge, this report has not been submitted, in whole or in part, for the award of any other degree or diploma.

Mayank Raj (241210076)

Md Yahiya Ariz (241210077)

Sahil Dindwal (241210102)

Yash Chawliya (241210135)

Signatures and Date: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. Gautam Kumar, for the guidance, suggestions and encouragement provided during the development of this project. Their feedback helped us understand the technical and practical aspects of crowd density estimation and public-safety monitoring.

We are thankful to the Department of Computer Science and Engineering, National Institute of Technology Delhi, for providing the academic environment and resources required to undertake this work. We also acknowledge the authors of CSRNet, the ShanghaiTech Crowd Counting Dataset contributors, and the developers of Python, PyTorch, OpenCV, NumPy, SciPy, h5py, Matplotlib, Pandas and Jupyter Notebook.

Finally, we thank our families, classmates and friends for their support and motivation throughout the project.

Mayank Raj Md Yahiya Ariz

Sahil Dindwal Yash Chawaliya

ABSTRACT

Large public gatherings at railway stations, bus terminals, stadiums, religious venues and public events can develop unsafe crowd concentrations within a short period. Manual monitoring of multiple camera feeds is demanding and may delay the identification of congestion. This project presents a deep-learning-based prototype for estimating crowd density from images and translating the estimated count into a simple public-safety warning level.

The system is based on CSRNet, a convolutional neural network designed for crowd counting in highly congested scenes. Its frontend uses VGG-16 feature extraction and its backend uses dilated convolutions to preserve spatial resolution while increasing the receptive field. The network produces a continuous density map; summing the density values provides an estimated crowd count. The project uses the ShanghaiTech Crowd Counting Dataset, including Part A and Part B. Ground-truth density maps are generated from head-point annotations using geometry-adaptive Gaussian kernels for Part A and a fixed Gaussian kernel for Part B. The PyTorch data pipeline applies ImageNet normalization and reduces target density maps by a factor of eight in each spatial dimension while applying a 64-fold scale to preserve the count integral.

The notebook includes a checkpoint-loading inference pipeline, count-error and latency measurement functions, heatmap visualization, an interactive Jupyter widget, and batch analysis. In the recorded Part B experiment, 20 test images were processed with a reported aggregate predicted count of 2,666, comprising 4 SAFE, 9 WARNING and 7 CRITICAL image-level classifications. The notebook reports an average inference latency of 3.3 ms (approximately 305.8 FPS) for that batch on an NVIDIA GeForce RTX 2050. A separate four-image Part A batch reported an aggregate predicted count of 1,426 and average latency of 8.7 ms (approximately 114.9 FPS). These are notebook batch measurements and should not be interpreted as end-to-end live-camera system throughput.

For response demonstration, a serial actuator class maps SAFE, WARNING and CRITICAL states to G, Y and R commands for an Arduino/ESP32-compatible receiver. The recorded run used mock mode; therefore, physical siren, strobe and turnstile actuation is proposed through the interface and firmware, but is not claimed as physically verified. The report documents the problem, related work, methodology, system analysis, design, implementation, prototype testing, results, limitations and future scope.

Keywords: Crowd Counting, Crowd Density Estimation, CSRNet, VGG-16, Dilated Convolution, Density Map, ShanghaiTech Dataset, Public Safety, Heatmap, Arduino, ESP32.

TABLE OF CONTENTS

Sr. No.	Contents	Page No.
1	Introduction	1
2	Literature Review	3
3	Methodology	5
4	System Analysis	9
5	System Design	11
6	Implementation	15
7	Prototype Integration and Deployment Considerations	19
8	Testing	21
9	Results and Discussion	23
10	Conclusion and Future Scope	31
References	Bibliography	33

LIST OF FIGURES

- Fig. 1.1 Overall project workflow
- Fig. 3.1 Project methodology
- Fig. 5.1 Proposed system architecture
- Fig. 5.2 Data Flow Diagram — Level 0
- Fig. 5.3 Data Flow Diagram — Level 1
- Fig. 5.4 CSRNet architecture
- Fig. 5.5 Notebook module organization
- Fig. 5.6 IoT actuator interface
- Fig. 9.1 Part B batch safety distribution
- Fig. 9.2 Part A batch safety distribution

LIST OF TABLES

- Table 1.1 Project objectives
- Table 3.1 Methodology phases
- Table 4.1 Hardware requirements
- Table 4.2 Software requirements
- Table 5.1 Dataset summary
- Table 5.2 CSRNet layer summary
- Table 6.1 Notebook module descriptions
- Table 6.2 Training configuration defined in code
- Table 8.1 Functional test cases
- Table 8.2 Integration test cases
- Table 9.1 Part B batch results
- Table 9.2 Part A batch results
- Table 9.3 Part B safety summary
- Table 9.4 Part A safety summary

1. INTRODUCTION

1.1 Background

Crowd gatherings are common in transport hubs, religious places, sports venues, markets, festivals and public demonstrations. A large number of people sharing a limited space can create bottlenecks and unsafe local concentrations. The risk is influenced not only by the total number of people but also by their spatial distribution, movement direction, available exits and the physical layout of the venue. A monitoring system that can estimate crowd concentration from visual input can support operators in identifying locations that may require attention.

Conventional CCTV systems provide video feeds to human operators. However, monitoring many feeds continuously is labour-intensive and subject to fatigue, distraction and delayed reaction. Object detectors that locate each person with a bounding box may also face difficulty in extremely dense scenes because individuals overlap and become partially or fully occluded.

Crowd counting through density-map regression offers an alternative. Instead of detecting a separate bounding box for every person, a model estimates a continuous spatial density field. The sum or integral of this field approximates the number of people in the scene. CSRNet is a convolutional neural network architecture designed for this task, using a VGG-based frontend and a dilated-convolution backend.

1.2 Problem Statement

The project addresses the problem of estimating the number and spatial concentration of people in a crowd image and converting the estimate into an understandable safety status. The prototype accepts image input, preprocesses it, applies a trained CSRNet checkpoint, generates a density map, integrates the map to obtain a count, and compares the count with configurable warning and critical thresholds. The output includes a numerical estimate, a heatmap and a SAFE, WARNING or CRITICAL status.

1.3 Objectives

- Develop a crowd-density estimation pipeline using CSRNet and PyTorch.
- Prepare ShanghaiTech Part A and Part B data using annotated head points and density maps.
- Implement geometry-adaptive and fixed Gaussian density-map generation.
- Estimate crowd count by integrating the predicted density map.
- Measure count error and inference latency on selected test images.
- Visualize spatial crowd concentration using density heatmaps and overlays.
- Classify image-level crowd estimates into SAFE, WARNING and CRITICAL levels.
- Demonstrate batch monitoring and a serial interface for Arduino/ESP32-compatible alert hardware.

1.4 Scope of the Project

The implemented scope is a research and demonstration prototype centred on image-based inference in a Jupyter Notebook. It includes dataset preparation utilities, CSRNet model definition, checkpoint loading, optional training code, evaluation functions, visual analysis, an interactive widget, batch image telemetry and a mockable serial actuator interface. The recorded experiments use selected ShanghaiTech test images.

The notebook does not demonstrate a production CCTV stream ingestion service, a continuously running web dashboard, a calibrated venue occupancy system, or verified physical turnstile control. Such functions are discussed as possible extensions. The safety thresholds are configurable demonstration values and must be calibrated to venue capacity and operational procedures before any real-world use.

1.5 Motivation

The project is motivated by the need to turn camera imagery into actionable information that can assist human crowd managers. A density heatmap can reveal where people are concentrated, while an estimated count provides a compact summary. Combining the estimate with threshold-based alert levels demonstrates how machine-learning output can be connected to an operational response workflow.

1.6 System Workflow Overview

The overall pipeline begins with a camera image or stored image file. The image is converted to RGB, normalized using ImageNet statistics and passed to CSRNet. The model returns a one-channel density map at reduced spatial resolution. Summing its values yields the estimated count. The density map is resized for display and colour-mapped to create a heatmap. A threshold engine assigns the safety status and corresponding action text. The batch module records results in a Pandas table, while the actuator class can print simulated telemetry or send serial commands when physical serial mode is explicitly enabled.

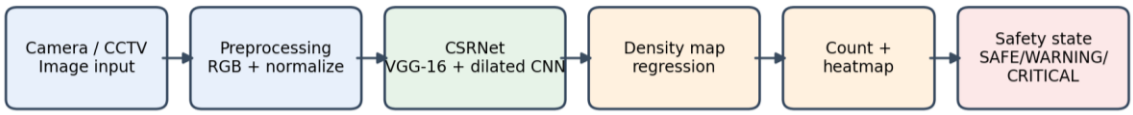


Fig. 1.1 Overall Project Workflow

1.7 Organization of the Report

Chapter 2 reviews crowd-counting approaches and the CSRNet foundation. Chapter 3 explains the methodology and dataset preparation. Chapter 4 analyses the existing problem, proposed system and feasibility. Chapter 5 presents the architecture, data-flow diagrams, dataset and model design. Chapter 6 describes the notebook implementation. Chapter 7 discusses prototype integration and deployment considerations. Chapter 8 presents test cases. Chapter 9 discusses recorded results and limitations. Chapter 10 concludes the work and identifies future enhancements.

2. LITERATURE REVIEW

2.1 Crowd Counting Approaches

Crowd counting research broadly includes detection-based, regression-based and density-estimation approaches. Detection-based methods attempt to locate individual people or body parts. They can be effective in sparse scenes but may lose accuracy when people overlap heavily. Regression-based methods learn a direct mapping from image features to a global count, but a single scalar output does not describe where the crowd is concentrated. Density-map approaches estimate a spatial map whose integral corresponds to the count, providing both a count and spatial information.

2.2 Density Map Regression

A density map represents each annotated person by a localized distribution rather than a hard bounding box. If head annotations are represented by points x_i, a Gaussian kernel is placed around each point. The resulting map is the sum of the individual kernels. In an ideal construction, each point contributes approximately unit mass, so the map integral corresponds to the number of annotated heads.

$$D(p) = \Sigma_i \, \delta(p - x_i) * G\sigma_i(p) \quad \text{and} \quad C = \iint D(p)\, dp$$

This representation is particularly useful in dense scenes because it does not require the model to separate overlapping full-body silhouettes. The heatmap also provides a spatial explanation of the model's estimated concentration.

2.3 CSRNet: Dilated Convolutional Neural Network

CSRNet, introduced by Yuhong Li, Xiaofan Zhang and Deming Chen, addresses crowd counting in highly congested scenes using a two-part architecture. The frontend extracts features using the early layers of VGG-16. The backend uses dilated convolutions to increase the receptive field without adding further spatial downsampling. A final convolution produces a single-channel density map.

The key design choice is the use of dilated convolution. A dilation rate greater than one spaces kernel elements apart, allowing the layer to aggregate information from a wider region while retaining the resolution of its input feature map. This is valuable for crowd scenes where local head patterns and broader spatial context both matter.

2.4 ShanghaiTech Crowd Counting Dataset

The ShanghaiTech dataset is used in the notebook as the benchmark source for crowd-counting experiments. Part A contains highly congested scenes with large variation in density. Part B contains comparatively less congested street scenes. Images are accompanied by MATLAB annotation files containing head-point coordinates. The notebook extracts these coordinates and converts them into density maps.

2.5 Density Map Generation and Gaussian Kernels

The notebook applies two density-map strategies. For Part A, it uses a KD-Tree to obtain the three nearest non-self neighbours for each annotation and adapts the Gaussian spread to local point spacing. For Part B, it uses a fixed Gaussian standard deviation of 15 pixels. The Part A code bounds sigma between 1 and 30 pixels to avoid extremely small or large kernels.

$$\sigma_i = 0.1 \times (d_{i1} + d_{i2} + d_{i3}) = 0.3 \times mean(d_{i1}, d_{i2}, d_{i3})$$

The expression above is the implementation used in the notebook. It is equivalent to beta 0.3 times the mean of the three nearest-neighbour distances.

2.6 Research Gap and Project Positioning

The project focuses on connecting a crowd-counting model to a simple operational monitoring prototype. In addition to density estimation, it includes image-level safety categories, a multi-image telemetry table and a serial command interface for a microcontroller. The emphasis is on demonstrating an end-to-end concept from visual input to a possible alert response, rather than claiming a complete commercial crowd-management platform.

2.7 Summary

The reviewed concepts justify the selection of density regression and CSRNet for congested scenes. The dataset annotations provide point-level supervision, while Gaussian density maps create a continuous target. The project extends the model output with threshold-based interpretation and an IoT-oriented interface.

3. METHODOLOGY

3.1 Overview of the Methodology

The project follows a modular experimental workflow. The steps include problem study, dataset inspection, density-map construction, PyTorch data loading, CSRNet definition, checkpoint loading, optional training, evaluation, interactive visualization, batch testing and actuator simulation. Each phase produces an output consumed by the next phase.

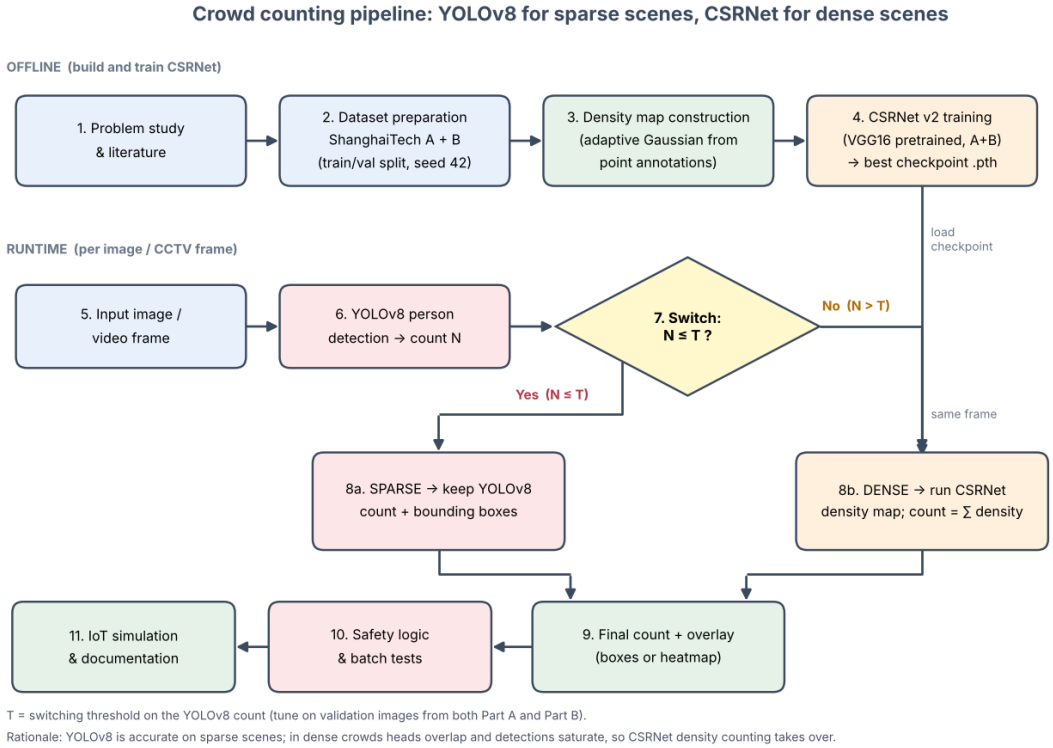


Fig. 3.1 Project Methodology

Phase	Activity	Tools / Libraries	Output
1	Problem definition and study of crowd-counting methods	Research paper, documentation	Problem scope and objectives
2	Load ShanghaiTech Part A and Part B images and annotations	SciPy, h5py, glob	Image and annotation paths
3	Generate continuous density maps	NumPy, SciPy KDTree, OpenCV	Ground-truth density arrays
4	Create dataset and data loaders	PyTorch, torchvision, PIL	Normalized image and density tensors
5	Define CSRNet and load checkpoint	PyTorch, torchvision VGG-16	Density prediction model
6	Optional fine-tuning and validation	Adam, MSELoss	Updated model checkpoint if run
7	Evaluate counts and inference timing	PyTorch, NumPy	Count errors and latency
8	Visualize predictions and safety levels	Matplotlib, OpenCV, ipywidgets	Heatmaps and interactive result
9	Batch monitoring and serial actuator simulation	Pandas, pyserial interface	Telemetry table and alert commands

3.2 Environment Setup and Reproducibility

The notebook imports Python libraries for file handling, numerical operations, image processing, dataset loading, visualization and deep learning. It sets Python, NumPy and PyTorch random seeds to 42. When CUDA is

available, all CUDA random generators are also seeded. The device selection logic chooses CUDA when available and otherwise falls back to CPU.

The saved execution output reports PyTorch 2.6.0+cu124, CUDA execution, an NVIDIA GeForce RTX 2050 GPU and 4.00 GB of available VRAM. These details describe the recorded development environment and may differ on another machine.

3.3 Data Acquisition and Preparation

The notebook expects the ShanghaiTech folder hierarchy to contain image files under images directories, annotation files under ground-truth directories, and optionally precomputed HDF5 density maps. The CrowdDataset class discovers JPG files, loads each image in RGB format and looks for a matching HDF5 file. If a density file is absent, it derives the MATLAB annotation path and constructs a density map from the head points.

3.4 Ground-Truth Density Map Construction

For each annotated head point, the generator places a Gaussian kernel in a local window around the annotation. The Gaussian is normalized by its continuous two-dimensional normalization factor. In Part A, a KD-Tree calculates neighbour distances and the local sigma is clipped to the interval [1, 30]. In Part B, sigma is fixed at 15.0. Out-of-image coordinates are skipped. The generated map is stored as a float32 NumPy array.

$$G\sigma(x,y) = (1 / 2\pi\sigma^2) \exp(-((x-x_i)^2 + (y-y_i)^2) / 2\sigma^2)$$

3.5 Image Preprocessing and Dataset Pipeline

Each image is converted to a tensor and normalized using the ImageNet channel means [0.485, 0.456, 0.406] and standard deviations [0.229, 0.224, 0.225]. During training, the dataset class applies a random horizontal flip with probability one-half and flips the corresponding density map to preserve alignment.

CSRNet's frontend performs three spatial 2×2 pooling operations, reducing the height and width by a factor of eight. The ground-truth density map is resized to H/8 by W/8 using cubic interpolation and multiplied by 64. This scale factor compensates for the 8×8 reduction in pixel count and is intended to preserve the total density integral.

$$D_s = 64 \times Resize(D, W/8, H/8)$$

3.6 Model Training Methodology

The notebook contains a complete training routine. It uses an MSELoss with sum reduction, the Adam optimizer with learning rate 1×10^{-5} and weight decay 1×10^{-4}, and a StepLR scheduler configured with step size 5 and gamma 0.5. The training function defaults to five epochs, while the notebook's example invocation specifies three epochs. The best checkpoint is selected using validation MAE.

Important implementation status: the saved notebook sets run_training = False. It reports 400 Part B training images and 316 validation images, but the optional training loop was not executed in the saved run. Instead, the model section reports loading checkpoints/best_model_part_B.pth. Accordingly, this report describes the available training pipeline but does not claim that the attached notebook run trained a new checkpoint or provide unrecorded epoch curves.

3.7 Evaluation Methodology

The evaluation functions calculate the predicted count by summing all values in the predicted density map and calculate the ground-truth count by summing the target density map. They accumulate absolute count error and squared count error across the evaluated samples. The square root of the mean squared count error is reported by the code under the label MSE; mathematically, this is root mean squared error (RMSE). The notebook also measures inference latency and converts average latency into an approximate frames-per-second value.

$$MAE = (1/N)\ \Sigma_i\ |Cpred_i - Cgt_i|$$

$$RMSE = \sqrt{[(1/N)\ \Sigma_i\ (Cpred_i - Cgt_i)^2]}$$

$$FPS \approx 1000\ /\ average\ latency\ in\ milliseconds$$

3.8 Safety Classification Methodology

The decision engine maps a predicted count to one of three states. In the interactive widget, the warning and critical thresholds can be changed using sliders. In the Part B batch test, the thresholds are 80 and 150 people. In the Part A batch test, they are 200 and 500 people. The thresholds differ because the experiments represent different crowd-density regimes; they are demonstration parameters, not universal safety limits.

Condition	Status	Demonstration response
Count < T_warning	SAFE	Normal monitoring; normal flow message
`T_warning ≤ Count < T_critical`	WARNING	Throttle ingress and alert ground marshals
`Count ≥ T_critical`	CRITICAL	Emergency message; siren and ingress-gate response proposed

3.9 Visualization and Multi-Image Monitoring

The notebook resizes the predicted density map to the original image dimensions, normalizes it for display, applies the OpenCV JET colour map and blends it with the original image. A coloured border indicates the safety status. Batch analysis records filename, predicted count, ground truth, absolute error, latency, throughput, safety level and an action message in a Pandas DataFrame.

3.10 Hardware and IoT Integration Methodology

The CrowdSafetyActuator class supports mock mode and a serial mode. In mock mode, it prints the command telemetry without opening a serial port. In physical mode, it attempts to open the configured COM port at 9600 baud and writes G, Y or R commands. If the connection fails, the class falls back to mock mode. The supplied Arduino sketch interprets these commands and controls four output pins.

3.11 Summary

The methodology combines dataset-driven density estimation with practical monitoring logic. The implementation supports offline image experiments and a mockable hardware communication pathway. Continuous live-camera processing and physical actuator validation remain outside the recorded experimental evidence.

4. SYSTEM ANALYSIS

4.1 Existing System

In many public venues, crowd monitoring depends on CCTV observation by security personnel. This approach can be effective when the number of cameras is limited and operators are attentive, but it becomes difficult to scale across multiple camera views. Visual judgement may vary between operators, and a rapid rise in local density can be difficult to quantify consistently.

Generic object detection systems can estimate people by detecting individual instances. In dense scenes, however, occlusion and overlap can cause missed detections or merged detections. Crowd density regression offers an alternative by predicting a continuous density field.

4.2 Proposed System

The proposed prototype processes an image through CSRNet, estimates a density map and count, visualizes high-density regions, and assigns a safety level based on configurable thresholds. It provides an interactive Jupyter interface for single-image analysis and a batch pipeline for multiple images. A serial actuator abstraction demonstrates how the resulting state can be communicated to an Arduino/ESP32-compatible receiver.

4.3 Feasibility Study

4.3.1 Technical Feasibility

The implementation uses established Python libraries and a pretrained-model architecture pattern. PyTorch provides tensor operations and neural-network layers; torchvision supplies VGG-16; OpenCV handles image operations and heatmap overlays; SciPy provides MATLAB file reading and KD-Tree neighbour queries; h5py reads density maps; and ipywidgets supports interactive controls. The recorded environment successfully used an NVIDIA RTX 2050 GPU.

4.3.2 Economic Feasibility

The software stack is based primarily on open-source libraries and a publicly used academic dataset. The notebook can be executed on a compatible personal computer with sufficient memory and optional CUDA-capable GPU acceleration. Physical hardware costs would depend on the selected microcontroller, LEDs, buzzer, power supply, drivers and safe installation. No actual hardware procurement or deployment cost is recorded in the notebook.

4.3.3 Operational Feasibility

The interactive widget is intended for a technical operator using Jupyter Notebook. The user selects an image, adjusts thresholds and runs the analysis. The batch function provides a tabular summary for a set of images. A production operator-facing interface would require a separate web or desktop application, camera connectivity, authentication, logging and operational training.

4.4 Hardware Requirements

Component	Requirement / Recorded Configuration
Processor	Intel Core i5 or equivalent recommended
RAM	8 GB minimum suggested; 16 GB preferred for dataset work
GPU	CUDA-capable NVIDIA GPU recommended; recorded run used RTX 2050 with 4 GB VRAM

Storage	Space for ShanghaiTech images, annotations, HDF5 maps, model checkpoint and Python environment
Camera	CCTV, drone or stored image source; live camera streaming not implemented in the saved notebook
Microcontroller	Arduino UNO or ESP32-compatible board for physical serial demonstration
Actuators	Green LED, amber LED, red strobe and piezo siren; driver circuitry and safe power design required

4.5 Software Requirements

Component	**Software / Library**
Language	Python
Deep learning	PyTorch 2.6.0+cu124 in recorded environment; torchvision
Image processing	OpenCV, Pillow
Scientific computing	NumPy, SciPy, h5py
Data handling	Pandas
Visualization	Matplotlib, ipywidgets, IPython display
Development environment	Jupyter Notebook / compatible Python environment
Hardware communication	pyserial (required only for physical serial mode)
Firmware	Arduino C++ sketch for compatible microcontroller

4.6 Functional Requirements

- Load crowd images and corresponding ground-truth annotations or density maps.
- Normalize input images before model inference.
- Generate or load density targets for dataset experiments.
- Load the CSRNet model checkpoint and predict a density map.
- Calculate estimated crowd count by summing density values.
- Generate a visual heatmap and overlay.
- Classify the predicted count using configurable thresholds.
- Process a batch of images and create a telemetry table.
- Simulate alert output and optionally transmit serial commands.

4.7 Non-Functional Requirements

- Performance: inference should be sufficiently fast for the intended monitoring interval.
- Reliability: invalid image paths and unavailable serial hardware should not silently be treated as successful operation.
- Usability: count, heatmap and safety status should be understandable to operators.
- Maintainability: model, preprocessing, evaluation and actuator functions should remain logically separated.
- Portability: CPU fallback should remain available where CUDA is unavailable.
- Safety: threshold decisions must be reviewed by human operators and must not be treated as a standalone guarantee of crowd safety.

5. SYSTEM DESIGN

5.1 System Architecture

The system can be viewed as five functional layers: input acquisition, image preprocessing, density estimation, safety interpretation and response presentation. The model layer contains the CSRNet checkpoint. The monitoring layer converts model output into a count and a heatmap. The response layer assigns a state and provides an action message or serial command.

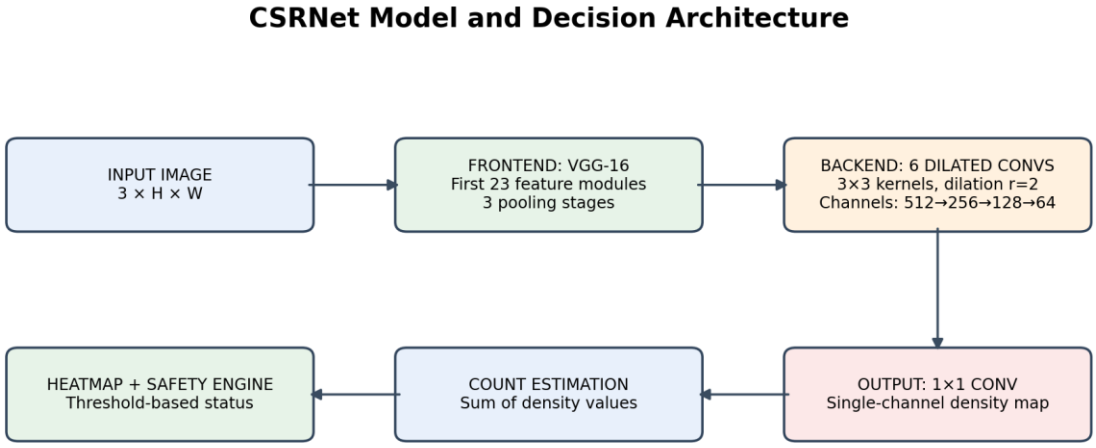


Fig. 5.1 Proposed System and CSRNet Architecture

5.2 Data Flow Diagram: Level 0

At the highest level, the external input is an image supplied by a camera or an image file. The system returns a crowd-monitoring result containing an estimated count, density heatmap and safety status. In the current notebook, the operator interacts with the system through Jupyter.

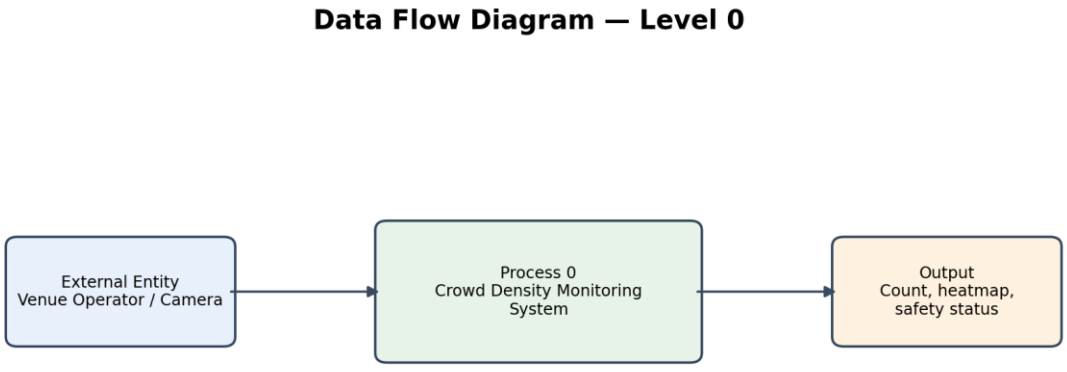


Fig. 5.2 Data Flow Diagram — Level 0

5.3 Data Flow Diagram: Level 1

The Level 1 diagram decomposes the monitoring process into image preparation, CSRNet inference, density integration, safety classification and output reporting. The trained checkpoint is used by the inference process. The output can be displayed visually and optionally passed to the actuator interface.

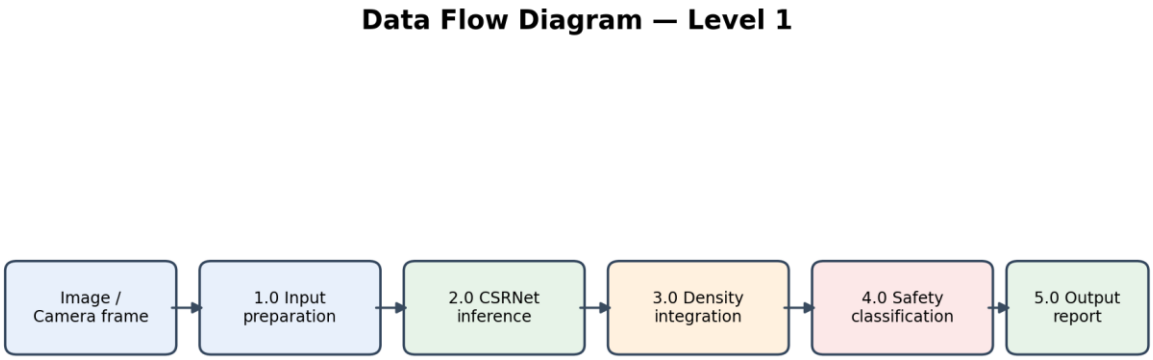


Fig. 5.3 Data Flow Diagram — Level 1

5.4 Dataset Description

Dataset Part	Scene Characteristics	Density-map strategy	Recorded test use
ShanghaiTech Part A	Highly congested scenes with strong density variation	KD-Tree geometry-adaptive Gaussian; sigma clipped to 1–30	First 4 test images in recorded batch
ShanghaiTech Part B	Comparatively less congested street scenes	Fixed Gaussian sigma = 15.0	20 test images in recorded batch; 316 test images available

The notebook's Part B training and validation loader output reports 400 training images and 316 validation images. The batch test uses 20 images from the sorted Part B test-image list. The Part A experiment uses four images. Dataset counts and sample selections are those printed by the submitted notebook.

5.5 CSRNet Model Architecture

The model is implemented as a PyTorch nn.Module. The frontend is constructed from the first 23 modules of torchvision's VGG-16 feature sequence. This portion contains the convolutional feature extractor through the third pooling stage and produces 512-channel features at one-eighth input resolution. The backend contains six 3×3 convolutional layers with dilation rate 2 and padding 2. Channel widths are 512, 512, 512, 256, 128 and 64, with ReLU activations after each convolution. A final 1×1 convolution maps the 64 channels to a single density channel.

Fig. 5.4 CSRNet Feature Extraction and Density Regression

Stage	Configuration	Purpose
Input	Input	Input
RGB image (3 channels)	RGB image (3 channels)	RGB image (3 channels)
Visual input to the network	Visual input to the network	Visual input to the network

Frontend	Frontend	Frontend
VGG-16 feature modules 0–22	VGG-16 feature modules 0–22	VGG-16 feature modules 0–22
Hierarchical feature extraction; output stride 8	Hierarchical feature extraction; output stride 8	Hierarchical feature extraction; output stride 8
Backend 1–3	Backend 1–3	Backend 1–3

5.6 Count Estimation

The predicted count is calculated by summing all values in the model's output density map. Since the model predicts a continuous field rather than discrete detections, the result may be a floating-point value. The notebook clamps the batch-test count to a minimum of zero before displaying it.

$$\hat{C} = \Sigma_r \Sigma_c \hat{D}(r,c)$$

5.7 Safety Threshold Design

The status engine applies two thresholds, T_warning and T_critical. The notebook uses 80 and 150 for Part B batch analysis and 200 and 500 for Part A. The interactive widget exposes sliders for threshold adjustment. Thresholds should be configured based on the physical area, venue capacity, camera coverage, crowd movement and safety policy.

5.8 Notebook and Module Organization

The complete implementation is provided in one Jupyter Notebook rather than a multi-file Python package. Its logical modules are environment setup, dataset preparation, density-map generation, dataset loading, model definition, optional training, evaluation, interactive monitoring, batch testing and actuator integration.

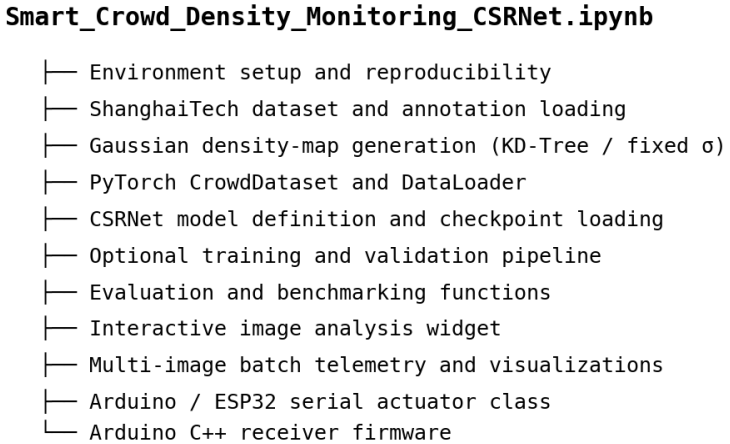


Fig. 5.6 Notebook Module Organization

6. IMPLEMENTATION

6.1 Development Environment Setup

The notebook imports os, sys, time, glob, math and random for runtime operations; NumPy, SciPy and Pandas for numerical and tabular work; OpenCV, Pillow and h5py for image and density-map handling; Matplotlib and ipywidgets for visualization; and PyTorch with torchvision for model construction and inference. The random seed is set to 42. The device is selected dynamically as CUDA or CPU.

6.2 Dataset Module: CrowdDataset

The CrowdDataset class searches for JPG images in the selected data directory. It derives the corresponding HDF5 path by replacing the image extension. When an HDF5 density map is present, it loads the dataset named density. Otherwise, it derives a MATLAB annotation path, reads image_info and extracts the head-point coordinates. It then generates the density map using the appropriate Part A or Part B strategy.

6.3 Density Generation Module

The generate_gaussian_density function initializes a float32 array with the same height and width as the image. If there are no annotations, it returns an all-zero map. For Part A with more than three points, a KDTree query returns each point's distance to itself and its three nearest neighbours. The three non-zero distances determine sigma. For Part B, a fixed sigma is used. A local Gaussian kernel is evaluated in a window of approximately three sigma around each annotation and accumulated into the map.

6.4 Data Augmentation and Tensor Conversion

Random horizontal flipping is applied only when the dataset is in training mode. The image and density map are flipped together. The image is transformed to a tensor and normalized with ImageNet statistics. The density map is resized to one-eighth of its original height and width, multiplied by 64 and returned as a single-channel tensor alongside the image tensor and image path.

6.5 CSRNet Model Module

The CSRNet class initializes a VGG-16 model and extracts the first 23 feature modules. In the submitted notebook, the model is instantiated with pretrained_frontend=False. The notebook then checks for checkpoints/best_model_part_B.pth and loads its state dictionary if the file exists. The recorded output confirms that the checkpoint was found and loaded. The backend and output-layer weights are initialized from a normal distribution with standard deviation 0.01, with biases set to zero; these initializations are relevant when a checkpoint is not loaded.

6.6 Training Module

The training function sets the model to training mode, performs forward propagation, computes the summed MSE loss, backpropagates gradients and updates parameters using Adam. A learning-rate scheduler reduces the learning rate after each five-epoch interval. After each epoch, validation computes loss, MAE and RMSE. If validation MAE improves, the state dictionary is saved to the checkpoint path.

The example training cell sets run_training=False. The training code is therefore available but was not run in the saved notebook execution. This distinction is maintained in the report so that no unsupported training curves or newly trained model claim is introduced.

6.7 Evaluation Module

The evaluation routine uses torch.no_grad() to disable gradient computation during inference. It calculates the predicted and ground-truth counts, accumulates absolute and squared differences, records inference latency and computes average throughput. The code's variable named mse is square-rooted before return, so the returned statistic is RMSE even though the printed label says MSE. This report uses the mathematically accurate name RMSE when explaining the formula.

6.8 Interactive Monitoring Interface

The run_interactive_widget function creates a dropdown containing up to 20 Part B test images, a warning-threshold slider, a critical-threshold slider, a button and an output area. When the button is pressed, the selected image is read and normalized, the model predicts a density map, and the sum gives the predicted count. The widget displays the estimated count, available ground truth, latency, throughput, safety badge and recommended action. It then renders the input image, density heatmap and blended safety overlay.

6.9 Batch Monitoring Module

The test_multiple_images function processes each supplied image sequentially. For each image, it prepares the normalized tensor, runs inference, sums the density map and attempts to locate a ground-truth count from an HDF5 map or MATLAB annotation. It classifies the image using the provided thresholds and appends a record to a DataFrame. For a limited number of images, it also displays the original image with a status border, the heatmap and a blended overlay.

6.10 Serial Actuator Module

The CrowdSafetyActuator class accepts a serial port, baud rate and mock flag. With mock=True, it prints timestamped telemetry. With mock=False, it imports pyserial and attempts to connect to the specified port. The trigger_alert method maps the status to a command byte and writes it when a physical serial connection is open. The class includes a close method to release the connection.

6.11 Arduino Firmware

The supplied Arduino sketch initializes pins 10 through 13 as outputs and starts serial communication at 9600 baud. It reads incoming characters and changes the outputs according to the command. The code is a firmware example; the saved notebook's recorded actuator test was performed in mock mode, not on a confirmed physical board.

```
G = SAFE      → Green LED (Pin 10)
Y = WARNING   → Amber LED (Pin 11)
R = CRITICAL  → Piezo siren (Pin 13) + red strobe (Pin 12)
Serial link   → 9600 baud
```

6.12 Source Module Summary

Logical Module	Main Functions / Classes	Responsibility
Environment Setup	`set_seed()`, device detection	Ensures reproducibility and selects the available computing device (CPU/GPU).
Density Map Generation	`generate_gaussian_density()`	Converts annotated head points into continuous Gaussian density maps.
Dataset Management	`CrowdDataset`	Loads images and density-map targets, performs data augmentation and normalization.

Model Architecture	`CSRNet`	Implements the VGG-16 frontend, dilated convolution backend and density-map output layer.
Training	`validate()`, `train_csrnet()`	Trains the CSRNet model, validates performance, applies optimization and selects checkpoints.
Evaluation	`evaluate_and_plot()`	Evaluates crowd-count errors, inference latency and visual predictions.
Interactive Interface	`run_interactive_widget()`	Supports single-image analysis and adjustment of safety thresholds.
Batch Processing	`test_multiple_images()`	Processes multiple images and generates evaluation metrics, telemetry and visual reports.
Safety Actuator	`CrowdSafetyActuator`	Handles mock or serial-based transmission of crowd-safety alerts.

7. PROTOTYPE INTEGRATION AND DEPLOYMENT CONSIDERATIONS

7.1 Prototype Integration Overview

The submitted implementation is an integrated notebook prototype. Dataset processing, model inference, visualization, batch reporting and actuator simulation are executed in the same Python environment. This design simplifies academic demonstration and makes the complete processing sequence inspectable in one file.

7.2 Input and Model Integration

The dataset and inference functions use a common image-normalization convention based on ImageNet channel statistics. The model is loaded from the Part B checkpoint path when available. During batch inference, the input image is converted from OpenCV's BGR representation to RGB before normalization. The model output is reduced to a scalar estimate by summation.

7.3 Monitoring and Reporting Integration

The interactive widget provides a user-driven image analysis path. The batch function provides a repeated analysis path for a list of image files. Both use the same model and count integration principle, while threshold values can differ according to the experiment. Batch telemetry is returned as a Pandas DataFrame and includes a text description of the proposed response.

7.4 Hardware Communication Integration

The actuator class provides an abstraction between the software decision and the microcontroller. In mock mode, it demonstrates the status-to-command mapping without hardware. In physical mode, the serial port must be configured correctly, the board must run compatible firmware, and the electrical outputs must be connected through appropriate drivers. The notebook does not verify those physical conditions.

7.5 Proposed Real-Time Deployment Architecture

A future deployment could place one inference service near the camera network or at an edge computer. Camera frames would be sampled at a configured interval, passed through preprocessing and CSRNet, and converted into count and heatmap outputs. A monitoring service would aggregate camera-level states, store timestamped telemetry and notify authorized personnel. A separate hardware gateway could receive signed or validated commands. This architecture is proposed future work; it is not part of the current notebook implementation.

7.6 Operational and Safety Considerations

- Calibrate thresholds using venue-specific floor area, safe occupancy limits and crowd-flow plans.
- Use a human-in-the-loop process for emergency decisions; model estimates may be wrong.
- Avoid directly locking gates solely from one image prediction. Gate control requires fail-safe design, manual override and emergency egress compliance.
- Protect camera feeds and stored telemetry with access control and retention policies.
- Monitor model drift caused by camera angle, lighting, weather, image compression and scene changes.
- Measure end-to-end latency, including capture, transfer, preprocessing, inference, decision and actuation—not only model forward-pass time.

7.7 Current Deployment Status

No hosted web service, browser dashboard, continuously running CCTV integration or physical IoT installation is documented in the notebook. The implemented deliverable is a Jupyter Notebook prototype with local dataset access, checkpoint-based inference, interactive widgets, batch analysis and mock actuator logic. A future deployment plan is included to show how the prototype could be extended without misrepresenting its current status.

8. TESTING

8.1 Testing Strategy

Testing in the notebook is based on functional execution of the data pipeline, checkpoint loading, inference, visualization, batch telemetry and actuator state mapping. The recorded notebook includes successful initialization messages, checkpoint loading output, batch summaries and mock actuator telemetry. The following test cases describe the functions and the expected behaviour; they are not a substitute for a formal independent test report.

8.2 Unit Test Cases

Test ID	Module	Test Input / Action	Expected Behaviour	Evidence / Status
UT-01	Density generator	Empty point array	Return all-zero density map	Implemented branch in code
UT-02	Density generator	Part A points	KD-Tree adaptive Gaussian maps	Function implemented
UT-03	Density generator	Part B points	Fixed sigma 15 Gaussian maps	Function implemented
UT-04	Dataset	Valid JPG with H5 target	Load RGB image and density map	Dataset class implemented
UT-05	CSRNet	Normalized tensor	Return one-channel density map	Checkpoint loaded in recorded run
UT-06	Count integration	Predicted density array	Sum values into scalar count	Used in evaluation and batch code
UT-07	Safety engine	Counts below / between / above thresholds	SAFE / WARNING / CRITICAL	Interactive and batch logic implemented
UT-08	Actuator	G, Y, R states in mock mode	Print matching pin/action telemetry	Mock scenarios recorded

8.3 Integration Test Cases

Test ID	Scenario	Expected Behaviour	Recorded Result
IT-01	Load Part B checkpoint	Model state dictionary is loaded	Notebook output confirms checkpoint loaded
IT-02	Part B batch run	Process selected 20 images and return table	Completed; summary printed
IT-03	Part A batch run	Process selected 4 images and return table	Completed; summary printed
IT-04	Heatmap overlay	Render input, heatmap and blended overlay	Figures generated for selected samples
IT-05	Mock actuator SAFE	Select green command	SAFE telemetry printed
IT-06	Mock actuator WARNING	Select amber command	WARNING telemetry printed
IT-07	Mock actuator CRITICAL	Select red command	CRITICAL telemetry printed
IT-08	Physical serial link	Connect to board and transmit commands	Not physically verified in recorded run

8.4 Dataset and Batch Testing

The notebook confirms 316 Part B test images were available and selects the first 20 sorted image paths for the recorded batch. It also selects the first four Part A test images. For each batch, it calculates estimates, compares them with ground-truth values when available, assigns safety levels and prints a venue-wide summary.

8.5 Limitations of Testing

The recorded batch tests use a small subset of each test partition rather than the complete benchmark. The notebook defines an evaluation function that can iterate over a DataLoader, but its printed benchmark output is not present in the saved execution. The saved training cell does not run training. Consequently, this report does not invent full-dataset MAE/RMSE values or epoch-wise learning curves. Latency figures from the batch function should be treated as measured notebook timings that may be affected by GPU synchronization and runtime overhead.

9. RESULTS AND DISCUSSION

9.1 Experimental Environment

The CSRNet training and evaluation experiments were performed using Python and Jupyter Notebook on a Windows-based system. The saved training notebook reports the following hardware and software configuration:

Parameter	Configuration
Deep learning framework	PyTorch 2.6.0+cu124
GPU acceleration	CUDA enabled
GPU	NVIDIA GeForce RTX 2050
GPU memory	4.00 GB
Model architecture	CSRNet
Frontend initialization	ImageNet-pretrained VGG-16
Operating environment	Windows, Python and Jupyter Notebook
Dataset	ShanghaiTech Crowd Counting Dataset
Checkpoint	`checkpoints/csrnet_v2_pretrained_AB_best.pth`

9.2 Dataset and Experimental Configuration

The experiments use Part A and Part B of the ShanghaiTech Crowd Counting Dataset. The notebook loads the training and testing images separately.

Dataset Split	Part A	Part B	Total
Training images	300	400	700
Validation images	30	40	70
Training images after validation split	270	360	630
Official test images	182	316	498

A 10% validation split is created from the training data using random seed 42. The official test folders remain separate from training and validation.

The main training configuration is shown below.

Parameter	Value
Input crop size	512 × 512
Batch size	1
Maximum configured epochs	80
Early stopping patience	12 epochs
Optimizer	Adam
Frontend learning rate	0.000001
Backend and output learning rate	0.0001
Weight decay	0.0001
Loss function	Summed pixel-wise Mean Squared Error
Gradient clipping	Maximum norm 10
Mixed precision	Enabled when CUDA is available

Learning rate scheduler	ReduceLROnPlateau

The training experiment uses ImageNet-pretrained VGG-16 features as the frontend and a dilated convolution backend for density-map prediction. The training and validation samples are drawn from both dataset domains.

9.3 Training Results

The notebook records seven complete training epochs and a partially executed eighth epoch. The configured maximum was 80 epochs, but the saved execution does not show completion of the full training process.

The model is selected using macro mean relative absolute error, which gives equal importance to the validation performance of Part A and Part B.

The recorded training results are:

Epoch	Training Loss	Part A Validation MAE	Part B Validation MAE	Macro Relative Error
1	45.8547	244.5135	51.8450	38.94%
2	38.3015	179.3913	32.4562	23.98%
3	36.2313	121.6280	20.1062	16.93%
4	31.2658	254.7204	106.6667	78.85%
5	30.8295	109.3822	12.9162	13.08%
6	27.0733	185.3864	19.0095	19.03%
7	33.6549	112.1953	16.7965	14.15%

Epoch 5 produced the best recorded validation result, with a macro relative error of approximately 13.08%.

The corresponding validation MAE values were:

- Part A: 109.38 people.
- Part B: 12.92 people.

The notebook saved the best checkpoint separately as csrnet_v2_pretrained_AB_best.pth.

Although eight epochs were started, the eighth epoch reached only 225 out of 630 training steps in the saved output. Therefore, the reported best result corresponds to epoch 5, not the final configured epoch.

9.4 Part A Test Results

The Part A CSV contains predictions for all 182 test images. Each record includes the predicted crowd count, ground-truth count, absolute error, reported accuracy, safety level, active engine and inference latency.

All 182 Part A records use the CSRNet Congested Density Engine.

The calculated evaluation metrics are:

Metric	Part A Result
Number of evaluated images	182
Mean predicted count	484.06
Mean ground-truth count	428.24
Mean Absolute Error (MAE)	104.56
Root Mean Squared Error (RMSE)	149.43
Mean signed error	+55.82
Mean relative error	26.52%
Median relative error	22.07%
Images within 10% relative error	20.88%
Images within 20% relative error	45.60%
Average inference latency	33.76 ms

The positive mean signed error indicates that the predictions tend to overestimate the ground-truth counts on this test split.

Part A Safety-Level Distribution

The CSV reports the following safety-level distribution:

Safety Level	Number of Images	Percentage
SAFE	4	2.20%
WARNING	51	28.02%
CRITICAL	127	69.78%
Total	**182**	**100%**

The results show that most Part A images are assigned the CRITICAL category by the application.

9.5 Part B Test Results

The Part B CSV contains predictions for all 316 test images.

Most records use the CSRNet Congested Density Engine, while 11 records use the YOLOv8 Sparse Detector. Therefore, the Part B results represent a mixed-engine evaluation rather than an exclusively CSRNet-based evaluation.

The calculated metrics are:

Metric	Part B Result
Number of evaluated images	316
Mean predicted count	121.55
Mean ground-truth count	116.21
Mean Absolute Error (MAE)	14.47
Root Mean Squared Error (RMSE)	23.36
Mean signed error	+5.33
Mean relative error	12.71%

Median relative error	9.32%
Images within 10% relative error	53.16%
Images within 20% relative error	83.23%
Average inference latency	5.89 ms

The positive mean signed error indicates a small overall tendency toward overestimation.

Active Engine Distribution

Active Engine	Number of Images
CSRNet Congested Density Engine	305
YOLOv8 Sparse Detector	11
Total	**316**

Part B Safety-Level Distribution

Safety Level	Number of Images	Percentage
SAFE	166	52.53%
WARNING	123	38.92%
CRITICAL	27	8.54%
Total	**316**	**100%**

The results show that the majority of Part B images are classified as SAFE or WARNING.

9.6 Sample-Level Prediction Analysis

The CSV files show that prediction errors vary across individual images. Some predictions are very close to the ground-truth counts, while others exhibit significant differences.

The following examples are taken directly from the uploaded evaluation CSV files.

Part A Examples

Image	Predicted Count	Ground Truth	Absolute Error	Safety Level
IMG_130.jpg	917.1	916.8	0.3	CRITICAL
IMG_95.jpg	184.0	185.4	1.4	WARNING
IMG_165.jpg	993.9	1576.5	582.7	CRITICAL
IMG_104.jpg	1700.8	1160.6	540.3	CRITICAL

IMG_130.jpg shows a very close prediction, with an absolute error of only 0.3 people. In contrast, IMG_165.jpg has a much larger error of approximately 582.7 people.

Part B Examples

Image	Predicted Count	Ground Truth	Absolute Error	Active Engine
IMG_50.jpg	250.3	250.3	0.0	CSRNet
IMG_14.jpg	28.4	28.4	0.1	CSRNet

IMG_17.jpg	197.0	64.8	132.2	CSRNet
IMG_80.jpg	339.0	440.3	101.4	CSRNet
IMG_1.jpg	20.0	21.9	1.9	YOLOv8

These examples demonstrate that the model can produce highly accurate estimates for some images, but substantial errors remain in other cases.

The IMG_1.jpg result also illustrates the use of the YOLOv8 engine for a sparse-scene prediction in Part B.

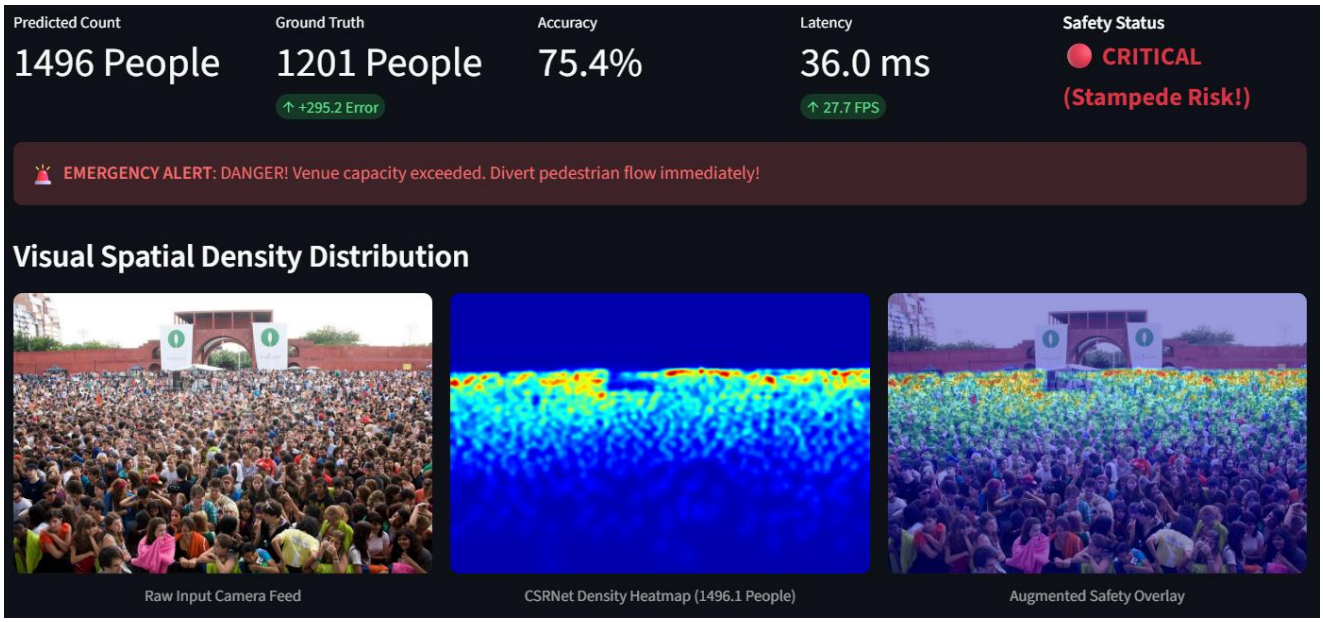


Figure 9.5: Example of a Part A prediction with a small counting error.



Figure 9.6: Example of a Part B prediction.

9.7 Quantitative Evaluation Metrics

CSRNet is a regression-based model that predicts a continuous density map. The estimated crowd count is obtained by summing the values of the predicted density map.

The following metrics are used to evaluate crowd-count estimation.

9.7.1 Mean Absolute Error (MAE)

MAE measures the average absolute difference between predicted and ground-truth crowd counts.

$$MAE = \frac{1}{N}\sum_{i=1}^{N}\left|C_i^{pred} - C_i^{gt}\right|$$

Where:

- N = Number of evaluated images.
- C_i^{pred} = Predicted count for image i.
- C_i^{gt} = Ground-truth count for image i.

A lower MAE indicates better average counting performance.

9.7.2 Root Mean Squared Error (RMSE)

RMSE measures the square root of the average squared prediction error.

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(C_i^{pred} - C_i^{gt}\right)^2}$$

RMSE penalizes larger prediction errors more heavily than MAE.

9.7.3 Mean Relative Error

Mean relative error expresses the average absolute prediction error as a proportion of the ground-truth count.

$$MRE = \frac{100}{N}\sum_{i=1}^{N}\frac{\left|C_i^{pred} - C_i^{gt}\right|}{\max\left(C_i^{gt}, 1\right)}$$

9.7.4 Inference Latency

Inference latency is the time taken by the active engine to process an input image. It is measured in milliseconds.

The CSV files record an average latency of 33.76 ms for Part A and 5.89 ms for Part B.

These values are based on the recorded evaluation outputs and should not automatically be interpreted as end-to-end video-processing performance.

9.7.5 Recorded Evaluation Summary

Metric	Part A	Part B	Combined
Number of images	182	316	498
MAE	104.56	14.47	47.40
RMSE	149.43	23.36	92.23
Mean relative error	26.52%	12.71%	17.76%
Median relative error	22.07%	9.32%	12.48%
Within 10% error	20.88%	53.16%	41.37%
Within 20% error	45.60%	83.23%	69.48%
Average latency	33.76 ms	5.89 ms	16.07 ms

The combined metrics are calculated over all 498 CSV records. They are not an average of the Part A and Part B metric values.

The lower Part B errors are an observed difference between the two test splits. They should not be interpreted as proof that crowd density alone determines model accuracy, since the two dataset parts also differ in their scene characteristics and the Part B CSV includes 11 YOLOv8 predictions.

9.8 Reported Accuracy and Classification Metrics

The CSV files report the following average accuracy values:

Dataset	Reported Accuracy
Part A	73.97%
Part B	87.90%
Combined	82.82%

The accuracy formula is not specified in the CSV files. Precision, recall, F1-score and confusion matrix are not calculated because independently labelled safety categories are unavailable.

9.9 Qualitative Results: Density Heatmaps

The CSRNet notebook provides visualizations of the original image, ground-truth density map and predicted density map. These heatmaps illustrate the model's estimated crowd distribution.

9.10 Discussion

The experiments demonstrate crowd-count estimation using CSRNet on ShanghaiTech Part A and Part B. The best recorded validation result was obtained at epoch 5, with a macro relative error of 13.08%.

Part A achieved an MAE of 104.56 and RMSE of 149.43, while Part B achieved an MAE of 14.47 and RMSE of 23.36. Some predictions were close to ground truth, whereas others showed substantial errors.

Part B includes 11 YOLOv8 predictions alongside 305 CSRNet predictions. Therefore, its metrics represent a mixed-engine evaluation. The CSV results are also not conclusively linked to the newly trained CSRNet v2 checkpoint.

9.11 Limitations

1. Training completed only seven full epochs; epoch 8 remained incomplete.
2. The final comparison between the original and new checkpoints is unavailable.
3. The exact checkpoint used for CSV predictions is not documented.
4. Part B contains predictions from both CSRNet and YOLOv8.
5. The reported accuracy formula is unspecified.
6. Independent ground-truth safety labels are unavailable.
7. Some images exhibit substantial counting errors.
8. Live CCTV performance and physical hardware deployment have not been validated.

9.12 Overall Result

The project demonstrates a CSRNet-based crowd-density monitoring pipeline using the ShanghaiTech dataset. The combined CSV evaluation covers 498 images and reports an MAE of 47.40 and RMSE of 92.23.

The results demonstrate the feasibility of crowd-count estimation and threshold-based safety monitoring. Further checkpoint comparison, complete training, safety-label validation and real-world testing are required before deployment.

10. CONCLUSION AND FUTURE SCOPE

10.1 Future Scope

- Run and document full training and validation experiments with saved epoch-wise loss, MAE and RMSE curves.
- Evaluate the complete official test partitions and report reproducible MAE/RMSE with confidence intervals or repeated runs where appropriate.
- Add frame-stream ingestion from CCTV or RTSP sources with frame sampling and reconnect handling.
- Build a web dashboard showing camera tiles, counts, heatmaps, timestamps, trends and alert history.
- Calibrate thresholds using venue-specific occupancy, area and safety procedures rather than generic fixed counts.
- Add temporal smoothing and alert persistence to reduce false alarms caused by one-frame fluctuations.
- Use camera calibration or perspective maps to estimate people per square metre and local density rather than relying only on total image count.
- Add multi-camera identity-aware aggregation or spatial overlap handling to avoid double counting across overlapping views.
- Validate the model under different camera angles, lighting conditions, weather and crowd types.
- Implement authenticated, encrypted communication with the microcontroller and add acknowledgement, watchdog and fail-safe behaviour.
- Conduct controlled physical hardware tests with manual override and emergency-egress safeguards.
- Explore edge deployment on an embedded GPU or accelerator and benchmark end-to-end latency and power consumption.

10.2 Conclusion

This project presents a CSRNet-based prototype for crowd density estimation and public-safety early warning. It uses VGG-16 feature extraction followed by a dilated-convolution backend to generate a continuous density map. The sum of the density values provides an estimated crowd count, while the spatial map supports heatmap visualization. The notebook prepares ShanghaiTech Part A and Part B annotations as density maps, provides a PyTorch dataset pipeline, loads a trained checkpoint, and includes optional training and evaluation functions.

The prototype extends crowd counting with a three-level safety decision engine, interactive single-image analysis, multi-image batch telemetry and an Arduino/ESP32-compatible serial interface. The recorded Part B batch processed 20 images and reported 2,666 estimated people with 4 SAFE, 9 WARNING and 7 CRITICAL classifications. The four-image Part A batch reported 1,426 estimated people with 1 SAFE and 3 WARNING classifications. Mock actuator tests demonstrated the G, Y and R command mapping.

The work demonstrates the feasibility of combining density-map regression with a simple warning workflow. However, the current deliverable remains an academic notebook prototype. Full benchmark evaluation, live camera streaming, venue-specific calibration, physical actuator validation and a production operator dashboard are necessary before deployment in a real public venue.

REFERENCES

[1] Y. Li, X. Zhang and D. Chen, "正 CSRNet: Dilated Convolutional Neural Networks for Understanding the Highly Congested Scenes," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 1091–1100.

[2] Y. Zhang, D. Zhou, S. Chen, S. Gao and Y. Ma, "Single-Image Crowd Counting via Multi-Column Convolutional Neural Network," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[3] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," International Conference on Learning Representations (ICLR), 2015.

[4] ShanghaiTech Crowd Counting Dataset, Part A and Part B, dataset and annotations used for academic crowd-counting research.

[5] PyTorch Documentation, https://pytorch.org/docs/.

[6] Torchvision Documentation, https://pytorch.org/vision/stable/.

[7] OpenCV Documentation, https://docs.opencv.org/.

[8] SciPy Documentation, https://docs.scipy.org/.

[9] NumPy Documentation, https://numpy.org/doc/.

[10] h5py Documentation, https://docs.h5py.org/.

[11] Matplotlib Documentation, https://matplotlib.org/stable/.

[12] Pandas Documentation, https://pandas.pydata.org/docs/.

[13] Arduino Language Reference, https://docs.arduino.cc/language-reference/.